

TypeScript Programming Foundations

5-Day Beginner TypeScript Course

Table of Contents (Facilitator)

- 1. [Facilitator Preparation](#)
- 2. [Day 1: TypeScript Basics - Facilitator Guide](#)
- 3. [Day 2: Advanced Types & Type Systems - Facilitator Guide](#)
- 4. [Day 3: Classes, Interfaces & OOP - Facilitator Guide](#)
- 5. [Day 4: Generics, Utility Types & Advanced Patterns - Facilitator Guide](#)
- 6. [Day 5: TypeScript Project & Real-World Applications - Facilitator Guide](#)
- 7. [Appendix: Answer Keys](#)
- 8. [Appendix: TypeScript Setup Guide](#)

Facilitator Preparation (TypeScript)

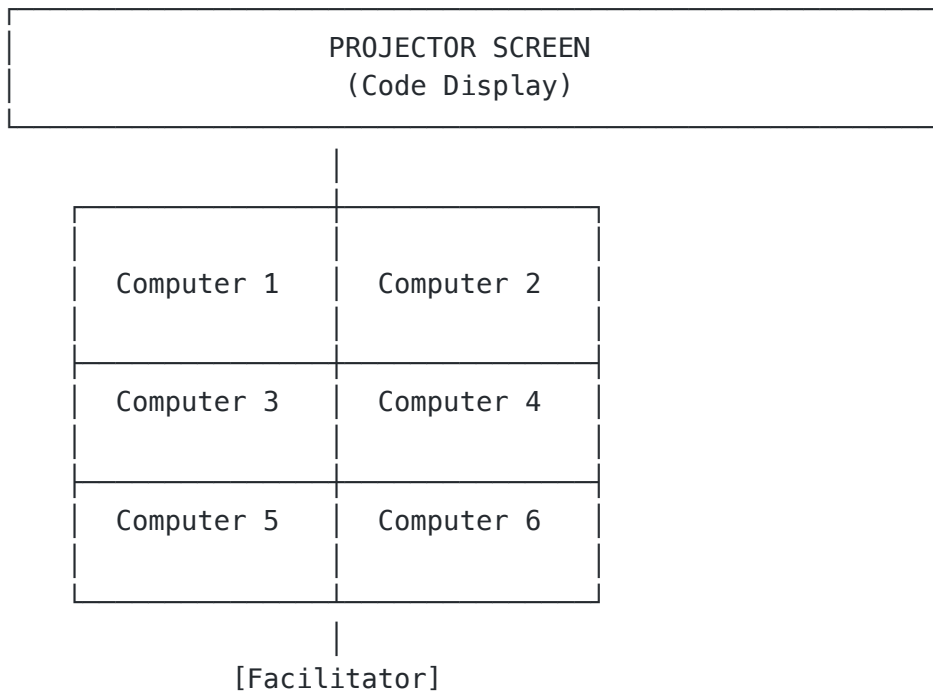
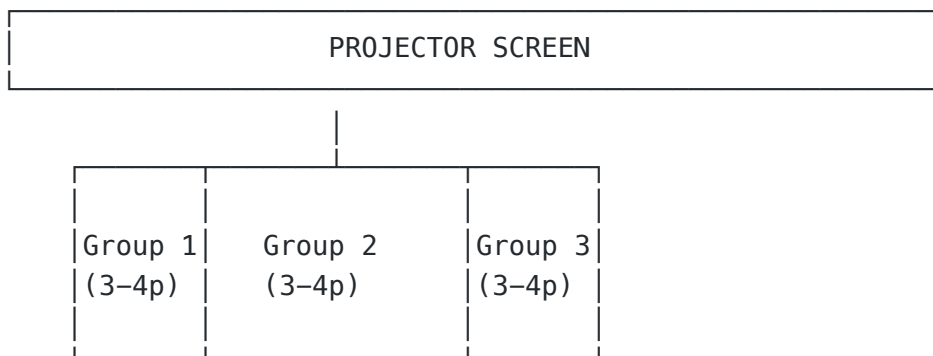
Master Props Checklist

Item	Quantity	Purpose	Day Used
Computers with Node.js installed	1 per participant	Running TypeScript	All
VS Code with TypeScript extension	1 per computer	Code editing	All
Projector/Display	1	Code demonstrations	All
Whiteboard + markers	1 large	Diagrams and concepts	All
Printed worksheets	1 set per participant	Exercises	All
Timer	1	Activity timing	All
TypeScript cheat sheets	1 per participant	Reference	All

TypeScript Installation Checklist

Before Day 1, ensure:

- [] Node.js 14+ installed on all computers
- [] TypeScript installed globally: `npm install -g typescript`
- [] VS Code with TypeScript extension installed
- [] Test that `tsc --version` works
- [] Test that `npx tsc` works
- [] `tsconfig.json` can be generated
- [] Internet access for additional resources

Classroom Setup**Day 1-4: Computer Lab Configuration****Day 5: Group Configuration**

|
[Facilitator]

Day 1: TypeScript Facilitator Guide

TypeScript Basics - Types, Variables & Functions

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Opening & TypeScript Setup Check	15 min
09:15-09:45	TypeScript Overview & Basic Types	30 min
09:45-10:15	EXERCISE: Type Annotations	30 min
10:15-10:45	Type Inference & Arrays/Tuples	30 min
10:45-11:15	EXERCISE: Arrays and Tuples	30 min
11:15-11:45	Functions with TypeScript	30 min
11:45-12:15	EXERCISE: Typed Functions	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Function Types & Callbacks	60 min
14:00-15:00	BUILD: Complete TypeScript Program	60 min
15:00-15:30	Day 1 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Opening & Setup Check

[SAY] "Good morning everyone. Welcome to TypeScript Programming Foundations. Before we start, let's verify everyone is set up correctly."

[CHECKLIST - Go through each item]

1. Check Node.js:

```
node --version
```

2. Check TypeScript:

```
tsc --version
```

3. Create a test file:

```
// test.ts
const message: string = "Hello TypeScript";
console.log(message);
```

4. Compile and run:

```
tsc test.ts
node test.js
```

[SAY] "If anyone is having issues, raise your hand. Once we're all set up, let's begin."

09:15 - 09:45 | TypeScript Overview & Basic Types

[WRITE ON WHITEBOARD]

Today's Goal: Understand TypeScript types
and write type-safe code.

[SAY] "TypeScript is JavaScript with types. Every valid JavaScript is valid TypeScript. The difference is that TypeScript catches errors before you run your code."

[PROJECT/DEMONSTRATE]

```
// JavaScript (runtime error)
function greet(name) {
  return "Hello, " + name;
}
greet(42); // No error until runtime - "Hello, 42" (not what you wanted!)
```

```
// TypeScript (compile-time error)
function greet(name: string): string {
  return "Hello, " + name;
}
greet(42); // Error: Argument of type 'number' is not assignable to parameter of type
```

[WRITE ON WHITEBOARD]

Basic Types

Type	Example
string	"Hello"
number	42 , 3.14
boolean	true , false
null	null
undefined	undefined

[DEMONSTRATE LIVE]

```
// String
let name: string = "Alice";

// Number
let age: number = 25;
let price: number = 19.99;

// Boolean
let isStudent: boolean = true;

// Null & Undefined
let empty: null = null;
let notDefined: undefined = undefined;
```

[SAY] "Key point: Type annotations are optional. TypeScript can infer types automatically."

```
// Type inference - no annotation needed
let name = "Alice"; // TypeScript knows it's a string
let age = 25;        // TypeScript knows it's a number
```

09:45 - 10:15 | EXERCISE: Type Annotations

[SAY] "Your turn. Add type annotations to these variables."

[PROJECT/WRITE]

```
// TODO: Add type annotations
let firstName = "John";
let age = 30;
```

```
let isActive = true;
let hobbies = ["reading", "gaming", "coding"];
let scores = [85, 90, 78];
let person = { name: "Alice", age: 25, city: "London" };
```

[GIVE 5 MINUTES]

[DEBRIEF]

```
let firstName: string = "John";
let age: number = 30;
let isActive: boolean = true;
let hobbies: string[] = ["reading", "gaming", "coding"];
let scores: number[] = [85, 90, 78];
let person: { name: string; age: number; city: string } = {
  name: "Alice",
  age: 25,
  city: "London"
};
```

[SAY] "Question: What happens if we do `firstName = 42` ? TypeScript will show an error because `firstName` is declared as `string`."

10:15 - 10:45 | Type Inference & Arrays/Tuples

[SAY] "TypeScript is smart enough to figure out types on its own. This is called **Type Inference**."

[DEMONSTRATE]

```
// Type inference examples
let message = "Hello"; // Type: string
let count = 42; // Type: number
let isReady = true; // Type: boolean
let items = [1, 2, 3]; // Type: number[]
let data = { id: 1, name: "test" }; // Type: { id: number; name: string; }
```

[SAY] "Arrays in TypeScript are typed. `number[]` means an array of numbers."

[WRITE ON WHITEBOARD]

Arrays

```
// Array of numbers
let numbers: number[] = [1, 2, 3, 4, 5];

// Array of strings
```

```
let fruits: string[] = ["apple", "banana", "cherry"];

// Array of mixed types (union)
let mixed: (string | number)[] = ["hello", 42, "world"];

// Alternative syntax
let moreNumbers: Array<number> = [6, 7, 8];
```

[SAY] "Tuples are arrays with a fixed number of elements, each with a specific type."

```
// Tuple - fixed length with specific types
let person: [string, number] = ["Alice", 25];

// Access
let name: string = person[0]; // "Alice"
let age: number = person[1];  // 25
```

[WRITE ON WHITEBOARD]

Tuples vs Arrays:

- Arrays: variable length, same type
- Tuples: fixed length, specific types per position

10:45 - 11:15 | EXERCISE: Arrays and Tuples

[SAY] "Practice with arrays and tuples."

[PROJECT/WRITE]

```
// 1. Create a string array for colors
let colors: string[] = ["red", "green", "blue"];

// 2. Create a number array for temperatures
let temperatures: number[] = [25.5, 30.2, 22.8];

// 3. Create a tuple for RGB color
let rgb: [number, number, number] = [255, 128, 0];

// 4. Create a tuple for a coordinate
let coordinate: [number, number] = [10, 20];
```

[GIVE 5 MINUTES]

[DEBRIEF] "Let's test these with some operations."

```
// Add to array
colors.push("yellow"); // Valid
// colors.push(42); // Error: Argument of type 'number' is not assignable to paramete

// Access tuple
let r: number = rgb[0]; // 255
let g: number = rgb[1]; // 128
let b: number = rgb[2]; // 0
```

11:15 - 11:45 | Functions with TypeScript

[SAY] "Functions in TypeScript have typed parameters and return types."

[WRITE ON WHITEBOARD]

```
// Function with typed parameters and return
function greet(name: string): string {
    return "Hello, " + name;
}

// Multiple parameters
function add(a: number, b: number): number {
    return a + b;
}

// Optional parameters (with ?)
function buildName(firstName: string, lastName?: string): string {
    if (lastName) {
        return firstName + " " + lastName;
    }
    return firstName;
}

// Default parameters
function greetWithDefault(name: string = "Guest"): string {
    return "Hello, " + name;
}
```

[DEMONSTRATE LIVE]

```
// Function with rest parameters
function sumAll(...numbers: number[]): number {
    return numbers.reduce((total, num) => total + num, 0);
}

console.log(sumAll(1, 2, 3, 4, 5)); // 15
```

```
// Void return (no return value)
function logMessage(message: string): void {
    console.log(message);
    // No return statement
}

// Never return (function never completes)
function throwError(message: string): never {
    throw new Error(message);
}
```

[WRITE ON WHITEBOARD]

Return Type	Description	Example
: string	Returns a string	return "Hello";
: number	Returns a number	return 42;
: boolean	Returns true/false	return true;
: void	No return value	console.log();
: never	Never returns	throw Error();

11:45 - 12:15 | EXERCISE: Typed Functions

[SAY] "Write these functions with proper TypeScript types."

[PROJECT/WRITE]

```
// 1. Function that doubles a number
function double(x: number): number {
    // Your code:
}

// 2. Function that greets a user
function greetUser(name: string, greeting?: string): string {
    // Your code:
}

// 3. Function that sums an array
function sumArray(numbers: number[]): number {
    // Your code:
}

// 4. Function that creates a person
function createPerson(name: string, age: number): { name: string; age: number } {
```

```
// Your code:  
}
```

[GIVE 10 MINUTES]

[DEBRIEF SOLUTIONS]

```
function double(x: number): number {  
    return x * 2;  
}  
  
function greetUser(name: string, greeting?: string): string {  
    return greeting ? `${greeting}, ${name}!` : `Hello, ${name}!`;  
}  
  
function sumArray(numbers: number[]): number {  
    return numbers.reduce((total, num) => total + num, 0);  
}  
  
function createPerson(name: string, age: number): { name: string; age: number } {  
    return { name, age };  
}
```

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Function Types & Callbacks

[SAY] "Functions can be treated as values. We can define function types."

[PROJECT/WRITE]

```
// Function type - what does this function look like?  
type Operation = (a: number, b: number) => number;  
  
// Use the function type  
let add: Operation = (a, b) => a + b;  
let multiply: Operation = (a, b) => a * b;  
  
// Function that accepts a callback  
function processNumbers(numbers: number[], callback: (num: number) => number): number[]  
    return numbers.map(callback);  
}  
  
// Test  
const numbers = [1, 2, 3, 4, 5];  
  
const doubled = processNumbers(numbers, (n) => n * 2);  
const squared = processNumbers(numbers, (n) => n * n);
```

```
console.log(doubled); // [2, 4, 6, 8, 10]
// [1, 2, 3, 4, 5] // [1, 2, 3, 4, 5] [1, 2, 3, 4, 5]
```

[SAY] "Exercise: Write a function that takes a callback and uses it."

[PROJECT/WRITE]

```
// Write a function 'processStrings' that takes an array of strings
// and a callback, and returns the processed array
function processStrings(strings: string[], callback: (str: string) => string): string[]
  // Your code:
}

// Test with:
// - Convert to uppercase
// - Add exclamation mark
// - Reverse each string
```

[GIVE 10 MINUTES]

[SOLUTION]

```
function processStrings(strings: string[], callback: (str: string) => string): string[]
  return strings.map(callback);
}

const names = ["alice", "bob", "charlie"];

const upper = processStrings(names, (str) => str.toUpperCase());
const excited = processStrings(names, (str) => str + "!");
const reversed = processStrings(names, (str) => str.split("").reverse().join(""));

console.log(upper); // ["ALICE", "BOB", "CHARLIE"]
console.log(excited); // ["alice!", "bob!", "charlie!"]
console.log(reversed); // ["ecila", "bob", "eilrahc"]
```

14:00 - 15:00 | BUILD: Complete TypeScript Program

[SAY] "Let's build a complete TypeScript program from scratch."

[PROJECT/WRITE]

Scenario: Student Grade Management System

Requirements:

1. Define a Student type with name, grades, and average
2. Create functions to:
 - Calculate average
 - Determine if passing (≥ 60)
 - Assign letter grade (A, B, C, D, F)
 - Generate a report
3. Test with sample data
4. Output the report

[GROUP EXERCISE - 30 minutes]

Teams of 3-4. Build the complete program.

[FACILITATOR SOLUTION]

```
// Define types
type Student = {
  name: string;
  grades: number[];
  average?: number;
  status?: string;
  letterGrade?: string;
};

// Functions
function calculateAverage(grades: number[]): number {
  const sum = grades.reduce((total, grade) => total + grade, 0);
  return sum / grades.length;
}

function isPassing(average: number): boolean {
  return average >= 60;
}

function getLetterGrade(average: number): string {
  if (average >= 90) return "A";
  if (average >= 80) return "B";
  if (average >= 70) return "C";
  if (average >= 60) return "D";
  return "F";
}

function processStudent(student: Student): Student {
  const avg = calculateAverage(student.grades);
  return {
    ...student,
    average: avg,
  };
}
```

```

        status: isPassing(avg) ? "Passing" : "Failing",
        letterGrade: getLetterGrade(avg)
    };
}

function processClass(students: Student[]): Student[] {
    return students.map(processStudent);
}

function generateReport(students: Student[]): string {
    let report = "Class Report\n";
    report += "=".repeat(40) + "\n\n";

    students.forEach(student => {
        report += `Name: ${student.name}\n`;
        report += `Grades: ${student.grades.join(", ")}\n`;
        report += `Average: ${student.average?.toFixed(2)}\n`;
        report += `Status: ${student.status}\n`;
        report += `Letter Grade: ${student.letterGrade}\n`;
        report += "-".repeat(20) + "\n";
    });

    return report;
}

// Main program
const students: Student[] = [
    { name: "Alice", grades: [85, 90, 78] },
    { name: "Bob", grades: [55, 60, 65] },
    { name: "Charlie", grades: [92, 88, 94] }
];

const processedStudents = processClass(students);
const report = generateReport(processedStudents);
console.log(report);

```

15:00 - 15:30 | DAY 1 WRAP-UP

[SAY] "Let's recap Day 1."

[WRITE ON WHITEBOARD - Fill in with group]

1. TypeScript = JavaScript with _____
2. Basic Types: ____, ____, ____, ____, ____
3. Type Inference: TypeScript figures out types _____
4. Arrays: _____[] or Array<_____>
5. Tuples: Fixed length with _____ types
6. Functions: _____ parameters and _____ return types

Answers:

1. Types
2. string, number, boolean, null, undefined
3. Automatically
4. T, T
5. Specific
6. Typed, typed

[SAY] "One question: **What surprised you most about TypeScript today?** "

[CLOSING STATEMENT] "Today we learned the basics of TypeScript - types, variables, and functions. Tomorrow we'll dive into advanced types and the type system."

[HOMEWORK] "Create a TypeScript file that:

1. Defines a Product interface with id, name, price, and inStock
2. Creates an array of products
3. Defines functions to calculate total price and find products by name"

Day 1 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting type annotations	"TypeScript can infer most types. Only annotate when needed."
Using <code>any</code> instead of proper types	"Avoid <code>any</code> . Use <code>unknown</code> if you must."
Array type syntax confusion	"Both <code>number[]</code> and <code>Array<number></code> work."
Tuple vs Array confusion	"Tuple = fixed length, specific types. Array = variable length, same type."
Not handling optional parameters	"Use <code>?</code> for optional: <code>lastName?: string</code> "

Day 2: TypeScript Facilitator Guide

Advanced Types & Type Systems

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 1 Recap	15 min
09:15-09:45	Union & Intersection Types	30 min
09:45-10:15	EXERCISE: Union Types	30 min
10:15-10:45	Literal Types & Type Aliases	30 min
10:45-11:15	EXERCISE: Type Aliases	30 min
11:15-11:45	Interfaces vs Type Aliases	30 min
11:45-12:15	EXERCISE: Interface Design	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	Type Guards & Type Assertions	60 min
14:00-15:00	EXERCISE: Complex Types	60 min
15:00-15:30	Day 2 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 1 Recap

[SAY] "Quick recap from yesterday. **What is TypeScript?** "

(Answer: JavaScript with types)

"What are the basic types? "

(Answer: string, number, boolean, null, undefined)

"What is type inference? "

(Answer: TypeScript figuring out types automatically)

"What's the difference between arrays and tuples? "

(Answer: Arrays = variable length, same type. Tuples = fixed length, specific types.)

"Excellent. Now we build on that."

09:15 - 09:45 | Union & Intersection Types

[SAY] "Union and intersection types allow you to combine types in powerful ways."

[WRITE ON WHITEBOARD]

Union Types (|)

"Union types allow a value to be one of several types."

```
// Value can be string OR number
let value: string | number;
value = "Hello"; // Valid
value = 42;      // Valid
// value = true; // Error: Type 'boolean' is not assignable to type 'string | number'

// Union type in a function
function formatValue(value: string | number): string {
    return `Value: ${value}`;
}
```

[DEMONSTRATE]

```
// Union types with arrays
let mixedArray: (string | number)[] = ["Hello", 42, "World", 100];

// Type narrowing
function process(value: string | number): void {
    if (typeof value === "string") {
        console.log(value.toUpperCase());
    } else {
        console.log(value.toFixed(2));
    }
}
```

[WRITE ON WHITEBOARD]

Intersection Types (&)

"Intersection types combine multiple types into one."

```
interface Person {
    name: string;
    age: number;
}

interface Employee {
    employeeId: number;
    department: string;
}
```

```
// Intersection: Person AND Employee
type EmployeePerson = Person & Employee;

const alice: EmployeePerson = {
  name: "Alice",
  age: 30,
  employeeId: 12345,
  department: "Engineering"
};
```

[SAY] "Key difference: Union = OR, Intersection = AND."

09:45 - 10:15 | EXERCISE: Union Types

[SAY] "Practice with union types."

[PROJECT/WRITE]

```
// 1. Define a union type for ID (string OR number)
type ID = string | number;

// 2. Create a function that accepts ID
function getMessage(id: ID): string {
  // Your code:
  // If ID is string: return "User ID: " + id
  // If ID is number: return "User ID: #" + id
}

// 3. Define a union type for Status
type Status = "loading" | "success" | "error" | "idle";

// 4. Create a function that returns a message based on status
function getStatusMessage(status: Status): string {
  // Your code:
}
```

[GIVE 10 MINUTES]

[SOLUTION]

```
type ID = string | number;

function getMessage(id: ID): string {
  if (typeof id === "string") {
    return `User ID: ${id}`;
  } else {
    return `User ID: #${id}`;
  }
}
```

```

    }
}

type Status = "loading" | "success" | "error" | "idle";

function getStatusMessage(status: Status): string {
    switch (status) {
        case "loading": return "Loading...";
        case "success": return "Operation successful!";
        case "error": return "An error occurred!";
        case "idle": return "Ready";
    }
}

```

[DEBRIEF] "Notice how TypeScript narrows the type inside the `if` block. This is called **type narrowing**."

10:15 - 10:45 | Literal Types & Type Aliases

[SAY] "Literal types allow you to specify exact values as types."

[WRITE ON WHITEBOARD]

Literal Types

```

// String literal types
type Direction = "up" | "down" | "left" | "right";
type Status = "pending" | "success" | "error";

let direction: Direction = "up"; // Valid
// direction = "forward"; // Error!

// Numeric literal types
type DiceValue = 1 | 2 | 3 | 4 | 5 | 6;
let roll: DiceValue = 4; // Valid

// Const assertions
let greeting = "hello" as const; // Type: "hello"
const config = {
    env: "production",
    port: 3000
} as const; // All properties become readonly literals

```

[WRITE ON WHITEBOARD]

Type Aliases

"Type aliases give a name to a type."

```
// Type alias for primitive
type ID = string | number;

// Type alias for object
type Person = {
  name: string;
  age: number;
  email?: string; // Optional
};

// Type alias for function
type GreetFunction = (name: string) => string;

// Using type aliases
let userId: ID = "abc123";
let person: Person = { name: "Alice", age: 25 };
let greet: GreetFunction = (name) => `Hello, ${name}!`;
```

10:45 - 11:15 | EXERCISE: Type Aliases

[SAY] "Create type aliases for these scenarios."

[PROJECT/WRITE]

```
// 1. Type alias for HTTP methods
type HttpMethod = // Your code:

// 2. Type alias for a User object
type User = // Your code:

// 3. Type alias for a function that doubles a number
type DoubleFunction = // Your code:

// 4. Type alias for a product with readonly id
type Product = // Your code:
```

[GIVE 10 MINUTES]

[SOLUTION]

```
type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";

type User = {
  id: number;
  name: string;
```

```
    email: string;
    age?: number;
};

type DoubleFunction = (x: number) => number;

type Product = {
    readonly id: number;
    name: string;
    price: number;
    category: string;
    inStock: boolean;
};
```

11:15 - 11:45 | Interfaces vs Type Aliases

[SAY] "Interfaces and type aliases are similar, but there are key differences."

[WRITE ON WHITEBOARD]

Feature	Type Alias	Interface	
Declaration	<code>type Person = { name: string; }</code>	<code>interface Person { name: string; }</code>	
Extends	Intersection: <code>type Child = Person & { age: number; }</code>	<code>interface Child extends Person</code>	
Union types	Yes: <code>`type Value = string </code>	<code>number;`</code>	No
Declaration merging	No	Yes	
Primary use	Union types, complex types	Object shapes, classes	

[DEMONSTRATE]

```
// Interface
interface Person {
    name: string;
    age: number;
}

// Interface extending another
interface Employee extends Person {
    employeeId: string;
    department: string;
}
```

```
// Interface with optional properties
interface Product {
  readonly id: number;
  name: string;
  price: number;
  description?: string; // Optional
}

// Type alias for union
type Status = "active" | "inactive" | "pending";

// Type alias for object
type Config = {
  env: "dev" | "prod";
  apiUrl: string;
  timeout: number;
};
```

[SAY] "Rule of thumb: Use interfaces for object shapes. Use type aliases for unions, tuples, and complex types."

11:45 - 12:15 | EXERCISE: Interface Design

[SAY] "Design interfaces for these scenarios."

[PROJECT/WRITE]

```
// 1. Design an interface for a BlogPost
interface BlogPost {
  // Your code:
}

// 2. Design an interface for a Comment
interface Comment {
  // Your code:
}

// 3. Design an interface for a User with address
interface User {
  // Your code:
}

interface Address {
  // Your code:
}
```

[GIVE 10 MINUTES]

[SOLUTION]

```
interface BlogPost {  
  id: number;  
  title: string;  
  content: string;  
  author: string;  
  tags: string[];  
  published: boolean;  
  createdAt: Date;  
  updatedAt?: Date;  
}
```

```
interface Comment {  
  id: number;  
  postId: number;  
  author: string;  
  content: string;  
  createdAt: Date;  
}
```

```
interface Address {  
  street: string;  
  city: string;  
  state: string;  
  zipCode: string;  
  country: string;  
}
```

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  address: Address;  
  age?: number;  
}
```

Afternoon Session Script

13:00 - 14:00 | Type Guards & Type Assertions

[SAY] "Type guards help TypeScript narrow down types. Type assertions tell TypeScript what type you know a value is."

[WRITE ON WHITEBOARD]

Type Guards

```
// typeof type guard
function processValue(value: string | number | boolean): string {
    if (typeof value === "string") {
        return value.toUpperCase();
    }
    if (typeof value === "number") {
        return value.toFixed(2);
    }
    return value ? "true" : "false";
}

// instanceof type guard
class Dog { bark(): void { console.log("Woof!"); } }
class Cat { meow(): void { console.log("Meow!"); } }

function handlePet(pet: Dog | Cat): void {
    if (pet instanceof Dog) {
        pet.bark();
    } else {
        pet.meow();
    }
}

// User-defined type guard
function isString(value: unknown): value is string {
    return typeof value === "string";
}

function process(value: unknown): void {
    if (isString(value)) {
        console.log(value.toUpperCase());
    }
}
```

[WRITE ON WHITEBOARD]

Type Assertions

```
// Using 'as' syntax
let someValue: unknown = "Hello TypeScript!";
let strLength: number = (someValue as string).length;

// DOM element assertion
const inputElement = document.getElementById("user-input") as HTMLInputElement;
inputElement.value = "New value";

// API response assertion
const response = await fetch("/api/data");
const data = await response.json() as { id: number; name: string };
```

[SAY] "Type assertions are a way to tell TypeScript 'Trust me, I know what type this is.' Use them sparingly."

[DEMONSTRATE]

```
// The 'never' type – for exhaustive checking
type Color = "red" | "green" | "blue";

function getColorHex(color: Color): string {
  switch (color) {
    case "red": return "#FF0000";
    case "green": return "#00FF00";
    case "blue": return "#0000FF";
    default:
      // Exhaustive check – ensures all cases handled
      const exhaustiveCheck: never = color;
      return exhaustiveCheck;
  }
}
```

14:00 - 15:00 | EXERCISE: Complex Types

[SAY] "Build complex types using everything we've learned."

[PROJECT/WRITE]

```
// 1. Define a type for a Shape (circle or rectangle)
type Shape =
  // Your code:

// 2. Define a Circle type
type Circle = {
  // Your code:
};

// 3. Define a Rectangle type
type Rectangle = {
  // Your code:
};

// 4. Create a function that calculates area
function calculateArea(shape: Shape): number {
  // Your code:
  // Use type guards to handle different shapes
}

// 5. Define a type for an API response
```

```
type ApiResponse<T> =  
  // Your code:
```

[GIVE 20 MINUTES]

[SOLUTION]

```
type Circle = {  
  kind: "circle";  
  radius: number;  
};  
  
type Rectangle = {  
  kind: "rectangle";  
  width: number;  
  height: number;  
};  
  
type Shape = Circle | Rectangle;  
  
function calculateArea(shape: Shape): number {  
  if (shape.kind === "circle") {  
    return Math.PI * shape.radius * shape.radius;  
  } else {  
    return shape.width * shape.height;  
  }  
}  
  
type ApiResponse<T> =  
  | { success: true; data: T }  
  | { success: false; error: string }  
  | { loading: true };  
  
// Test  
const circle: Circle = { kind: "circle", radius: 5 };  
const rectangle: Rectangle = { kind: "rectangle", width: 4, height: 6 };  
  
console.log(calculateArea(circle));    // 78.54  
console.log(calculateArea(rectangle)); // 24  
  
const response: ApiResponse<{ id: number; name: string }> = {  
  success: true,  
  data: { id: 1, name: "Alice" }  
};
```

15:00 - 15:30 | DAY 2 WRAP-UP

[SAY] "Let's recap Day 2."

[WRITE ON WHITEBOARD]

1. Union Type: **OR** – use ____
2. Intersection Type: **AND** – use ____
3. Literal Types: Specific values as types
4. Type Alias: ``type Name = Type;``
5. Interface: ``interface Name { }``
6. Type Guard: Narrow types with ____, ____, custom guards
7. Type Assertion: ``as Type``
8. Never: Values that ____ occur

Answers:

1. |
2. &
3. "success" | "error"
- 4.
- 5.
6. typeof , instanceof
- 7.
8. Never

[SAY] "One question: **What's the difference between a type alias and an interface?** "

(Answer: *Type aliases can do unions, interfaces can be extended and merged*)

[CLOSING STATEMENT] "Today we explored advanced types - unions, intersections, literals, and type guards. Tomorrow we'll dive into classes and object-oriented programming with TypeScript."

[HOMEWORK] "Create a TypeScript file that:

1. Defines a type for a Shape (discriminated union)
2. Implements calculateArea and calculatePerimeter functions
3. Uses type guards and exhaustive checking"

Day 2 Facilitator Cheat Sheet

Mistake	How to Fix	
Confusing `	and &`	" = OR, & = AND"
Overusing any	"Use unknown if you must, or better types"	

Mistake	How to Fix	
Forgetting type guards	"TypeScript needs help narrowing types"	
Using type alias when interface better	"Use interface for object shapes that can extend"	
Misusing type assertions	"Only use when you're absolutely sure"	

Day 3: TypeScript Facilitator Guide

Classes, Interfaces & Object-Oriented Programming

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 2 Recap	15 min
09:15-09:45	Classes in TypeScript	30 min
09:45-10:15	EXERCISE: Creating Classes	30 min
10:15-10:45	Access Modifiers & Readonly	30 min
10:45-11:15	EXERCISE: Access Modifiers	30 min
11:15-11:45	Inheritance & Interfaces	30 min
11:45-12:15	EXERCISE: Inheritance	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	Abstract Classes & Polymorphism	60 min
14:00-15:00	BUILD: Complete OOP System	60 min
15:00-15:30	Day 3 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 2 Recap

[SAY] "Quick recap. What's a union type? "

(Answer: `type` | `type - value` can be either)

"What's an intersection type? "

(Answer: type & type - combines both)

"What's a type guard? "

(Answer: A condition that narrows the type)

"What's the difference between type alias and interface? "

(Answer: Type aliases do unions; interfaces do extends/merge)

"Excellent. Today we learn object-oriented programming with TypeScript."

09:15 - 09:45 | Classes in TypeScript

[SAY] "Classes are blueprints for creating objects. TypeScript adds type safety to classes."

[WRITE ON WHITEBOARD]

Basic Class

```
class Person {
  // Properties
  name: string;
  age: number;

  // Constructor
  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }

  // Method
  greet(): string {
    return `Hello, I'm ${this.name} and I'm ${this.age} years old.`;
  }
}

// Create instance
const alice = new Person("Alice", 25);
console.log(alice.greet()); // "Hello, I'm Alice and I'm 25 years old."
```

[DEMONSTRATE]

```
// Parameter properties – shorthand
class Person {
  constructor(
```

```

    public name: string,
    public age: number,
    private id: string,
    protected role: string
  ) {
    // No need to assign manually
  }
}

// Getter and Setter
class Temperature {
  private _celsius: number = 0;

  get celsius(): number {
    return this._celsius;
  }

  set celsius(value: number) {
    if (value < -273.15) {
      throw new Error("Temperature below absolute zero");
    }
    this._celsius = value;
  }

  get fahrenheit(): number {
    return (this._celsius * 9/5) + 32;
  }
}

const temp = new Temperature();
temp.celsius = 25;
console.log(temp.celsius);    // 25
console.log(temp.fahrenheit); // 77

```

09:45 - 10:15 | EXERCISE: Creating Classes

[SAY] "Create a Book class."

[PROJECT/WRITE]

```

// Create a Book class with:
// - Properties: title (string), author (string), pages (number), isRead (boolean)
// - Constructor: initializes all properties
// - Method: read() - marks book as read
// - Method: getInfo() - returns book information

class Book {
  // Your code:
}

```

[GIVE 10 MINUTES]

[SOLUTION]

```
class Book {
  private title: string;
  private author: string;
  private pages: number;
  private isRead: boolean;

  constructor(title: string, author: string, pages: number) {
    this.title = title;
    this.author = author;
    this.pages = pages;
    this.isRead = false;
  }

  read(): string {
    this.isRead = true;
    return `You have read "${this.title}" by ${this.author}`;
  }

  getInfo(): string {
    const status = this.isRead ? "Read" : "Not read yet";
    return `"${this.title}" by ${this.author} - ${this.pages} pages - ${status}`;
  }
}

const book = new Book("1984", "George Orwell", 328);
console.log(book.getInfo()); // "1984" by George Orwell - 328 pages - Not read yet
console.log(book.read());    // "You have read "1984" by George Orwell"
```

10:15 - 10:45 | Access Modifiers & Readonly

[WRITE ON WHITEBOARD]

Access Modifiers

Modifier	Description
public	Accessible anywhere (default)
private	Accessible only within the class
protected	Accessible within class and subclasses
readonly	Can only be set at initialization

[DEMONSTRATE]

```
class BankAccount {
    public readonly accountNumber: string;
    private balance: number;
    protected owner: string;
    public type: string;

    constructor(accountNumber: string, initialBalance: number, owner: string) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.owner = owner;
        this.type = "checking";
    }

    deposit(amount: number): void {
        if (amount <= 0) throw new Error("Amount must be positive");
        this.balance += amount;
    }

    withdraw(amount: number): boolean {
        if (this.canWithdraw(amount)) {
            this.balance -= amount;
            return true;
        }
        return false;
    }

    private canWithdraw(amount: number): boolean {
        return amount > 0 && amount <= this.balance;
    }

    getBalance(): number {
        return this.balance;
    }
}
```

[SAY] "Key points:

- private - only this class can access
- protected - this class and subclasses can access
- readonly - set once, never change"

10:45 - 11:15 | EXERCISE: Access Modifiers**[SAY]** "Practice with access modifiers."**[PROJECT/WRITE]**

```
// Create a Vehicle class with:
// - readonly property: vin (string)
// - private property: mileage (number)
// - protected property: make (string)
// - public property: model (string)
// - Method: drive(km) - adds to mileage
// - Method: getMileage() - returns mileage
```

```
class Vehicle {
    // Your code:
}
```

[GIVE 10 MINUTES]

[SOLUTION]

```
class Vehicle {
    public readonly vin: string;
    private mileage: number;
    protected make: string;
    public model: string;

    constructor(vin: string, make: string, model: string) {
        this.vin = vin;
        this.make = make;
        this.model = model;
        this.mileage = 0;
    }

    drive(km: number): void {
        if (km <= 0) {
            throw new Error("Distance must be positive");
        }
        this.mileage += km;
        console.log(`Drove ${km}km. Total: ${this.mileage}km`);
    }

    getMileage(): number {
        return this.mileage;
    }
}

const car = new Vehicle("ABC123", "Toyota", "Camry");
car.drive(50); // "Drove 50km. Total: 50km"
console.log(car.getMileage()); // 50
// car.mileage = 100; // Error: Property 'mileage' is private
```

11:15 - 11:45 | Inheritance & Interfaces

[SAY] "Classes can extend other classes and implement interfaces."

[WRITE ON WHITEBOARD]

Inheritance

```
// Base class
class Animal {
  protected name: string;

  constructor(name: string) {
    this.name = name;
  }

  makeSound(): void {
    console.log("Some animal sound");
  }
}

// Derived class
class Dog extends Animal {
  private breed: string;

  constructor(name: string, breed: string) {
    super(name); // Call parent constructor
    this.breed = breed;
  }

  makeSound(): void {
    console.log(`${this.name} says: Woof!`);
  }

  fetch(): void {
    console.log(`${this.name} is fetching`);
  }
}
```

[WRITE ON WHITEBOARD]

Implementing Interfaces

```
interface User {
  id: number;
  name: string;
  login(): void;
}

class Customer implements User {
  id: number;
```

```
    name: string;

    constructor(id: number, name: string) {
        this.id = id;
        this.name = name;
    }

    login(): void {
        console.log(`${this.name} logged in`);
    }
}

// Multiple interfaces
interface Person {
    name: string;
}

interface Worker {
    employeeId: string;
}

class Employee implements Person, Worker {
    name: string;
    employeeId: string;

    constructor(name: string, employeeId: string) {
        this.name = name;
        this.employeeId = employeeId;
    }
}
```

11:45 - 12:15 | EXERCISE: Inheritance

[SAY] "Create a class hierarchy."

[PROJECT/WRITE]

```
// 1. Create a Vehicle base class
class Vehicle {
    // Your code:
}

// 2. Create a Car class that extends Vehicle
class Car extends Vehicle {
    // Your code:
}

// 3. Create a Motorcycle class that extends Vehicle
class Motorcycle extends Vehicle {
```

```
// Your code:  
}
```

[GIVE 10 MINUTES]

[SOLUTION]

```
class Vehicle {  
    protected make: string;  
    protected model: string;  
    protected year: number;  
  
    constructor(make: string, model: string, year: number) {  
        this.make = make;  
        this.model = model;  
        this.year = year;  
    }  
  
    getInfo(): string {  
        return `${this.year} ${this.make} ${this.model}`;  
    }  
}  
  
class Car extends Vehicle {  
    private doors: number;  
  
    constructor(make: string, model: string, year: number, doors: number) {  
        super(make, model, year);  
        this.doors = doors;  
    }  
  
    getInfo(): string {  
        return `${super.getInfo()} - ${this.doors} doors`;  
    }  
}  
  
class Motorcycle extends Vehicle {  
    private type: string;  
  
    constructor(make: string, model: string, year: number, type: string) {  
        super(make, model, year);  
        this.type = type;  
    }  
  
    getInfo(): string {  
        return `${super.getInfo()} - ${this.type}`;  
    }  
}
```

```
const car = new Car("Toyota", "Camry", 2022, 4);  
console.log(car.getInfo()); // "2022 Toyota Camry - 4 doors"
```

Afternoon Session Script

13:00 - 14:00 | Abstract Classes & Polymorphism

[SAY] "Abstract classes define templates. Polymorphism means using different types through a common interface."

[WRITE ON WHITEBOARD]

Abstract Classes

```
abstract class Shape {  
  protected color: string;  
  
  constructor(color: string) {  
    this.color = color;  
  }  
  
  // Abstract method - must be implemented  
  abstract getArea(): number;  
  
  // Concrete method - can be used as-is  
  getDescription(): string {  
    return `A ${this.color} shape`;  
  }  
}  
  
class Square extends Shape {  
  private side: number;  
  
  constructor(color: string, side: number) {  
    super(color);  
    this.side = side;  
  }  
  
  // Must implement getArea  
  getArea(): number {  
    return this.side * this.side;  
  }  
}  
  
// Cannot instantiate abstract class  
// const shape = new Shape("red"); // Error!
```

[WRITE ON WHITEBOARD]

Polymorphism

```
abstract class Shape {
  abstract getArea(): number;
}

class Circle extends Shape {
  constructor(private radius: number) {
    super();
  }

  getArea(): number {
    return Math.PI * this.radius * this.radius;
  }
}

class Rectangle extends Shape {
  constructor(private width: number, private height: number) {
    super();
  }

  getArea(): number {
    return this.width * this.height;
  }
}

// Polymorphic function
function printArea(shape: Shape): void {
  console.log(`Area: ${shape.getArea().toFixed(2)}`);
}

const shapes: Shape[] = [
  new Circle(5),
  new Rectangle(4, 6)
];

shapes.forEach(shape => printArea(shape));
// "Area: 78.54"
// "Area: 24.00"
```

14:00 - 15:00 | BUILD: Complete OOP System

[SAY] "Build a complete object-oriented system."

[PROJECT/WRITE]

Scenario: Employee Management System

Requirements:

1. Abstract Employee class with name, salary
2. Manager class extends Employee with teamSize
3. Developer class extends Employee with language
4. Each employee has getRole(), getBonus(), getInfo()
5. Display all employees with their information
6. Calculate total payroll

[GROUP EXERCISE - 30 minutes]

[FACILITATOR SOLUTION]

```
abstract class Employee {
    protected name: string;
    protected salary: number;

    constructor(name: string, salary: number) {
        this.name = name;
        this.salary = salary;
    }

    abstract getRole(): string;
    abstract getBonus(): number;

    getInfo(): string {
        return `${this.getRole(): ${this.name} - ${this.salary}`;
    }
}

class Manager extends Employee {
    private teamSize: number;

    constructor(name: string, salary: number, teamSize: number) {
        super(name, salary);
        this.teamSize = teamSize;
    }

    getRole(): string {
        return "Manager";
    }

    getBonus(): number {
        return this.salary * 0.2;
    }

    getInfo(): string {
        return `${super.getInfo()} - Managing ${this.teamSize} people`;
    }
}

class Developer extends Employee {
    private language: string;
```

```

    constructor(name: string, salary: number, language: string) {
        super(name, salary);
        this.language = language;
    }

    getRole(): string {
        return "Developer";
    }

    getBonus(): number {
        return this.salary * 0.1;
    }

    getInfo(): string {
        return `${super.getInfo()} - ${this.language} developer`;
    }
}

// Main program
const employees: Employee[] = [
    new Manager("Alice", 80000, 5),
    new Developer("Bob", 60000, "TypeScript"),
    new Developer("Charlie", 65000, "Python")
];

console.log("Employee Report");
console.log("=".repeat(40));
employees.forEach(emp => {
    console.log(emp.getInfo());
    console.log(`Bonus: ${emp.getBonus()}`);
    console.log("-".repeat(20));
});

const totalPayroll = employees.reduce((total, emp) => total + emp.salary, 0);
console.log(`Total Payroll: ${totalPayroll}`);

```

15:00 - 15:30 | DAY 3 WRAP-UP

[SAY] "Let's recap Day 3."

[WRITE ON WHITEBOARD]

1. Class: Blueprint for ____
2. Access Modifiers: ____, ____, ____, ____
3. Inheritance: ____ keyword, ____ for parent
4. Abstract Class: Cannot be ____
5. Polymorphism: Different types through ____ interface

Answers:

1. Objects
2. public, private, protected, readonly
3. extends, super
4. Instantiated
5. Same

[SAY] "One question: **What's the difference between an abstract class and an interface?** "

(Answer: Abstract classes can have implementation. Interfaces only define contracts.)

[CLOSING STATEMENT] "Today we learned classes, inheritance, interfaces, and polymorphism. Tomorrow we'll explore generics and advanced patterns."

[HOMEWORK] "Create a TypeScript file that:

1. Defines an abstract `Animal` class with `makeSound()`
2. Creates `Dog` , `Cat` , and `Bird` classes
3. Uses polymorphism to handle different animals"

Day 3 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting <code>super()</code> in constructor	"Must call parent constructor before using <code>this</code> "
Using <code>private</code> when <code>protected</code> needed	"Use <code>protected</code> if subclasses need access"
Trying to instantiate abstract class	"Abstract classes are templates, not objects"
Not implementing abstract methods	"All abstract methods must be implemented"
Forgetting method override	"Use <code>override</code> keyword for clarity"

Day 4: TypeScript Facilitator Guide

Generics, Utility Types & Advanced Patterns

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 3 Recap	15 min
09:15-09:45	Generics Basics	30 min
09:45-10:15	EXERCISE: Generic Functions	30 min
10:15-10:45	Generic Classes & Constraints	30 min
10:45-11:15	EXERCISE: Generic Classes	30 min
11:15-11:45	Utility Types	30 min
11:45-12:15	EXERCISE: Utility Types	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	Custom Utility Types & Mapped Types	60 min
14:00-15:00	EXERCISE: Advanced Type Patterns	60 min
15:00-15:30	Day 4 Wrap-Up	30 min

Morning Session Script

09:00 - 09:15 | Day 3 Recap

[SAY] "Quick recap. **What's a class?** "

(Answer: Blueprint for objects)

"What are the access modifiers? "

(Answer: public, private, protected, readonly)

"What's inheritance? "

(Answer: extends keyword, super() for parent)

"What's polymorphism? "

(Answer: Using different types through the same interface)

"Excellent. Today we learn generics."

09:15 - 09:45 | Generics Basics

[SAY] "Generics allow you to create reusable components that work with different types."

[WRITE ON WHITEBOARD]

Generic Functions

```
// Generic function with type parameter T
function identity<T>(value: T): T {
    return value;
}

// Usage
const num = identity<number>(42);    // Type: number
const str = identity<string>("Hello"); // Type: string

// Type inference - TypeScript figures it out
const inferred = identity(42);        // Type: number

// Multiple type parameters
function pair<T, U>(first: T, second: U): [T, U] {
    return [first, second];
}

const p = pair("Alice", 25); // Type: [string, number]
```

[DEMONSTRATE]

```
// Generic constraints with extends
function getLength<T extends { length: number }>(value: T): number {
    return value.length;
}

console.log(getLength("Hello"));    // 5
console.log(getLength([1, 2, 3]));  // 3
// console.log(getLength(42));      // Error!

// Generic with keyof
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const person = { name: "Alice", age: 25 };
console.log(getProperty(person, "name")); // "Alice"
// console.log(getProperty(person, "city")); // Error!
```

[WRITE ON WHITEBOARD]

Generic Interfaces

```
interface Box<T> {
  value: T;
  getValue(): T;
}

class NumberBox implements Box<number> {
  constructor(public value: number) {}
  getValue(): number { return this.value; }
}

class StringBox implements Box<string> {
  constructor(public value: string) {}
  getValue(): string { return this.value; }
}
```

09:45 - 10:15 | EXERCISE: Generic Functions

[SAY] "Write generic functions."

[PROJECT/WRITE]

```
// 1. Generic function that returns the first element
function firstElement<T>(array: T[]): T | undefined {
  // Your code:
}

// 2. Generic function that reverses an array
function reverseArray<T>(array: T[]): T[] {
  // Your code:
}

// 3. Generic function that filters with predicate
function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
  // Your code:
}

// 4. Generic function that finds an item
function findItem<T>(array: T[], predicate: (item: T) => boolean): T | undefined {
  // Your code:
}
```

[GIVE 10 MINUTES]

[SOLUTION]

```
function firstElement<T>(array: T[]): T | undefined {
  return array[0];
}
```

```

    }

    function reverseArray<T>(array: T[]): T[] {
        return array.slice().reverse();
    }

    function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
        return array.filter(predicate);
    }

    function findItem<T>(array: T[], predicate: (item: T) => boolean): T | undefined {
        return array.find(predicate);
    }

    // Test
    const numbers = [1, 2, 3, 4, 5];
    const names = ["Alice", "Bob", "Charlie"];

    console.log(firstElement(numbers)); // 1
    console.log(reverseArray(numbers)); // [5, 4, 3, 2, 1]
    console.log(filterArray(numbers, n => n > 3)); // [4, 5]
    console.log(findItem(names, n => n === "Bob")); // "Bob"

```

10:15 - 10:45 | Generic Classes & Constraints

[SAY] "Generic classes allow you to create reusable classes with type parameters."

[WRITE ON WHITEBOARD]

Generic Classes

```

class Stack<T> {
    private items: T[] = [];

    push(item: T): void {
        this.items.push(item);
    }

    pop(): T | undefined {
        return this.items.pop();
    }

    peek(): T | undefined {
        return this.items[this.items.length - 1];
    }

    isEmpty(): boolean {
        return this.items.length === 0;
    }
}

```

```

    size(): number {
        return this.items.length;
    }
}

// Usage
const numberStack = new Stack<number>();
numberStack.push(1);
numberStack.push(2);
console.log(numberStack.pop()); // 2

const stringStack = new Stack<string>();
stringStack.push("Hello");
stringStack.push("World");
console.log(stringStack.pop()); // "World"

```

[DEMONSTRATE]

```

// Generic class with constraint
class KeyValuePair<K extends string | number, V> {
    constructor(private key: K, private value: V) {}

    getKey(): K { return this.key; }
    getValue(): V { return this.value; }
}

// Generic type alias
type Result<T> = {
    success: boolean;
    data?: T;
    error?: string;
};

function fetchData<T>(url: string): Promise<Result<T>> {
    // Implementation
    return fetch(url)
        .then(res => res.json())
        .then(data => ({ success: true, data }))
        .catch(error => ({ success: false, error: error.message }));
}

```

10:45 - 11:15 | EXERCISE: Generic Classes

[SAY] "Create generic classes."

[PROJECT/WRITE]

```
// 1. Generic Queue class (FIFO)
class Queue<T> {
    private items: T[] = [];

    // enqueue: adds item to the end
    enqueue(item: T): void {
        // Your code:
    }

    // dequeue: removes and returns the first item
    dequeue(): T | undefined {
        // Your code:
    }

    // peek: returns first item without removing
    peek(): T | undefined {
        // Your code:
    }

    // size: returns number of items
    size(): number {
        // Your code:
    }

    // isEmpty: returns true if empty
    isEmpty(): boolean {
        // Your code:
    }
}

// 2. Generic KeyValuePair class
class KeyValuePair<K, V> {
    // Your code:
}
```

[GIVE 10 MINUTES]

[SOLUTION]

```
class Queue<T> {
    private items: T[] = [];

    enqueue(item: T): void {
        this.items.push(item);
    }

    dequeue(): T | undefined {
        return this.items.shift();
    }
}
```

```
    peek(): T | undefined {
        return this.items[0];
    }

    size(): number {
        return this.items.length;
    }

    isEmpty(): boolean {
        return this.items.length === 0;
    }
}

class KeyValuePair<K, V> {
    constructor(private key: K, private value: V) {}

    getKey(): K { return this.key; }
    getValue(): V { return this.value; }
    setValue(value: V): void { this.value = value; }
}

// Test
const queue = new Queue<string>();
queue.enqueue("First");
queue.enqueue("Second");
console.log(queue.dequeue()); // "First"
console.log(queue.size());    // 1

const pair = new KeyValuePair("name", "Alice");
console.log(pair.getKey());    // "name"
console.log(pair.getValue());  // "Alice"
```

11:15 - 11:45 | Utility Types

[SAY] "TypeScript provides built-in utility types for common transformations."

[WRITE ON WHITEBOARD]

Common Utility Types

Utility Type	Description	Example	
Partial<T>	Makes all properties optional	Partial<User>	
Required<T>	Makes all properties required	Required<Config>	

Utility Type	Description	Example	
Readonly<T>	Makes all properties readonly	Readonly<User>	
Pick<T, K>	Selects a subset of properties	<code>Pick<User, "id" \</code>	<code>"name"></code>
Omit<T, K>	Excludes a subset of properties	Omit<User, "password">	
Record<K, T>	Creates an object type	Record<string, number>	
Exclude<T, U>	Excludes types	Exclude<Status, "loading">	
Extract<T, U>	Extracts types	Extract<Status, "success">	
NonNullable<T>	Excludes null and undefined	<code>NonNullable<string \</code>	<code>null></code>

[DEMONSTRATE]

```

interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  age: number;
}

// Partial - all optional
type PartialUser = Partial<User>;
// { id?: number; name?: string; email?: string; password?: string; age?: number; }

// Pick - select specific
type UserBasic = Pick<User, "id" | "name">;
// { id: number; name: string; }

// Omit - exclude specific
type UserPublic = Omit<User, "password">;
// { id: number; name: string; email: string; age: number; }

// Readonly - all readonly
type ReadonlyUser = Readonly<User>;
// { readonly id: number; readonly name: string; ... }

// Record - key-value object

```

```
type StatusMessage = Record<"success" | "error" | "pending", string>;  
// { success: string; error: string; pending: string; }
```

11:45 - 12:15 | EXERCISE: Utility Types

[SAY] "Use utility types to transform these types."

[PROJECT/WRITE]

```
interface User {  
  id: number;  
  name: string;  
  email: string;  
  password: string;  
  age: number;  
  address: string;  
}  
  
// 1. Make all properties optional  
type OptionalUser = // Your code:  
  
// 2. Select only id, name, email  
type UserContact = // Your code:  
  
// 3. Exclude password and address  
type UserPublic = // Your code:  
  
// 4. Make all properties readonly  
type ReadonlyUser = // Your code:  
  
// 5. Create a record type for user preferences  
type UserPreferences = Record<string, boolean>;  
  
// 6. Exclude null and undefined from a union  
type NonNullableString = NonNullable<string | null | undefined>;
```

[GIVE 10 MINUTES]

[SOLUTION]

```
type OptionalUser = Partial<User>;  
  
type UserContact = Pick<User, "id" | "name" | "email">;  
  
type UserPublic = Omit<User, "password" | "address">;  
  
type ReadonlyUser = Readonly<User>;
```

```
type UserPreferences = Record<string, boolean>;
```

```
type NonNullableString = NonNullable<string | null | undefined>;
```

Afternoon Session Script

13:00 - 14:00 | Custom Utility Types & Mapped Types

[SAY] "Create your own utility types with mapped types."

[WRITE ON WHITEBOARD]

Mapped Types

```
// Make all properties nullable
type Nullable<T> = {
  [P in keyof T]: T[P] | null;
};

// Make all properties optional and readonly
type ReadonlyPartial<T> = {
  readonly [P in keyof T]?: T[P];
};

// Deep readonly (recursive)
type DeepReadonly<T> = {
  readonly [P in keyof T]: T[P] extends object ? DeepReadonly<T[P]> : T[P];
};

// Extract keys with value of a specific type
type KeysOfType<T, U> = {
  [P in keyof T]: T[P] extends U ? P : never;
}[keyof T];
```

[DEMONSTRATE]

```
interface User {
  id: number;
  name: string;
  email: string;
  age: number;
  address: {
    street: string;
    city: string;
    zip: string;
  };
}
```

```
// Make all nullable
type NullableUser = Nullable<User>;
// All properties become T | null

// Deep readonly
type DeepReadonlyUser = DeepReadonly<User>;
// address becomes readonly too

// Get string keys
type StringKeys = KeysOfType<User, string>;
// "name" | "email" | "street" | "city" | "zip"
```

[WRITE ON WHITEBOARD]

Conditional Types

```
// Conditional type: T extends U ? X : Y
type IsString<T> = T extends string ? true : false;

type Result1 = IsString<string>; // true
type Result2 = IsString<number>; // false

// Extract return type using infer
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;

function greet(name: string): string {
  return `Hello, ${name}!`;
}

type GreetReturn = ReturnType<typeof greet>; // string
```

14:00 - 15:00 | EXERCISE: Advanced Type Patterns

[SAY] "Create advanced types and use them."

[PROJECT/WRITE]

```
// 1. Create a type that makes all properties nullable
type Nullable<T> = {
  // Your code:
};

// 2. Create a type that extracts string properties
type StringKeys<T> = {
  // Your code:
}[keyof T];
```

```
// 3. Create a DeepReadonly type
type DeepReadonly<T> = {
  // Your code:
};

// 4. Create a type for API response
type ApiResponse<T> =
  // Your code:

// 5. Use the types
interface Person {
  id: number;
  name: string;
  age: number;
  email: string;
  address: {
    street: string;
    city: string;
  };
}

// Test your types
```

[GIVE 20 MINUTES]

[SOLUTION]

```
type Nullable<T> = {
  [P in keyof T]: T[P] | null;
};

type StringKeys<T> = {
  [P in keyof T]: T[P] extends string ? P : never;
}[keyof T];

type DeepReadonly<T> = {
  readonly [P in keyof T]: T[P] extends object ? DeepReadonly<T[P]> : T[P];
};

type ApiResponse<T> =
  | { success: true; data: T }
  | { success: false; error: string }
  | { loading: true };

// Test
interface Person {
  id: number;
  name: string;
  age: number;
  email: string;
  address: {
```

```

        street: string;
        city: string;
    };
}

type NullablePerson = Nullable<Person>;
type PersonStringKeys = StringKeys<Person>; // "name" | "email" | "street" | "city"
type DeepReadonlyPerson = DeepReadonly<Person>;

const response: ApiResponse<Person> = {
    success: true,
    data: {
        id: 1,
        name: "Alice",
        age: 25,
        email: "alice@example.com",
        address: { street: "123 Main St", city: "London" }
    }
};

if (response.success) {
    console.log(response.data.name);
}

```

15:00 - 15:30 | DAY 4 WRAP-UP

[SAY] "Let's recap Day 4."

[WRITE ON WHITEBOARD]

1. Generics: _____ parameters for reusable components
2. Generic Functions: `(value: T): T`
3. Generic Classes: `class Stack<T>`
4. Utility Types: _____, _____, _____, _____, _____
5. Mapped Types: `[P in keyof T]: ...`
6. Conditional Types: `T extends U ? X : Y`
7. Infer: Extract types with `infer R`

Answers:

1. Type

2.

3.

4. Partial, Required, Readonly, Pick, Omit

5.

6.

7.

[SAY] "One question: **What's a generic?** "

(Answer: A type parameter that makes components reusable)

[CLOSING STATEMENT] "Today we learned generics, utility types, and advanced patterns. Tomorrow we'll apply everything to a complete TypeScript project."

[HOMEWORK] "Create a TypeScript file that:

- 1. Defines a generic Repository class
- 2. Uses utility types for different data views
- 3. Creates custom utility types"

Day 4 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting generic syntax	" <T> before parameters"
Not using constraints	"Use extends to limit what types can be used"
Overusing any	"Use generics instead"
Confusing utility types	"Partial = optional, Pick = select, Omit = exclude"
Complex generic errors	"Simplify the type or break it down"

Day 5: TypeScript Facilitator Guide

TypeScript Project & Real-World Applications

Session Plan Overview

Time	Activity	Duration
09:00-09:15	Day 4 Recap	15 min
09:15-09:45	TypeScript Configuration	30 min
09:45-10:15	EXERCISE: tsconfig.json	30 min
10:15-10:45	TypeScript with React	30 min
10:45-11:15	EXERCISE: React with TypeScript	30 min

Time	Activity	Duration
11:15-11:45	TypeScript with Node.js	30 min
11:45-12:15	EXERCISE: Node.js with TypeScript	30 min
12:15-13:00	LUNCH	45 min
13:00-14:00	EXERCISE: Complete TypeScript Project	60 min
14:00-15:00	Project Presentations	60 min
15:00-15:30	Final Wrap-Up & Certificates	30 min

Morning Session Script

09:00 - 09:15 | Day 4 Recap

[SAY] "Quick recap. What are generics? "

(Answer: Type parameters for reusable components)

"Name three utility types. "

(Answer: Partial, Pick, Omit, Readonly, Required, Record)

"What's a mapped type? "

(Answer: Transforming properties with `[P in keyof T]`)

"What's the `infer` keyword for? "

(Answer: Extracting types within conditional types)

"Excellent. Today we put everything together."

09:15 - 09:45 | TypeScript Configuration

[SAY] "TypeScript projects are configured with `tsconfig.json` ."

[PROJECT/WRITE]

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "lib": ["ES2020", "DOM"],
    "outDir": "./dist",
```

```
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

[SAY] "Key options explained:"

[WRITE ON WHITEBOARD]

Option	Description
target	JavaScript version to compile to
module	Module system (commonjs, esnext)
strict	Enable all strict checks
noImplicitAny	Error on implicit any
strictNullChecks	Strict null checking
outDir	Output directory
rootDir	Source directory

[DEMONSTRATE]

```
# Initialize tsconfig.json
npx tsc --init

# Compile project
npx tsc

# Watch mode
npx tsc --watch

# Compile specific file
npx tsc src/index.ts --outDir dist
```

09:45 - 10:15 | EXERCISE: tsconfig.json

[SAY] "Configure a TypeScript project."

[PROJECT/WRITE]

```
# 1. Create project structure
mkdir typescript-project
cd typescript-project
npm init -y

# 2. Install TypeScript
npm install typescript --save-dev

# 3. Create tsconfig.json
npx tsc --init

# 4. Update tsconfig.json with:
# - target: ES2020
# - module: commonjs
# - strict: true
# - outDir: ./dist
# - rootDir: ./src

# 5. Create src/index.ts

# 6. Compile and run
npx tsc
node dist/index.js
```

[GIVE 10 MINUTES]

[CHECKLIST]

- ☐ Project created with `npm init`
- ☐ TypeScript installed
- ☐ `tsconfig.json` created
- ☐ Configuration updated correctly
- ☐ Source file created
- ☐ Compilation successful
- ☐ Run output visible

10:15 - 10:45 | TypeScript with React

[SAY] "TypeScript makes React components type-safe."

[WRITE ON WHITEBOARD]

React with TypeScript

```
// Component with props
interface UserProps {
  name: string;
  age: number;
  email?: string; // Optional
}

const UserCard: React.FC<UserProps> = ({ name, age, email }) => {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      {email && <p>Email: {email}</p>}
    </div>
  );
};

// Event handlers
const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {
  console.log(event.target.value);
};

const handleSubmit = (event: React.FormEvent<HTMLFormElement>) => {
  event.preventDefault();
  // Form submission
};

// useState with TypeScript
const [user, setUser] = useState<User | null>(null);
const [loading, setLoading] = useState<boolean>(false);

// Custom hook
function useFetch<T>(url: string): {
  data: T | null;
  loading: boolean;
  error: Error | null;
} {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(data => setData(data))
      .catch(err => setError(err))
      .finally(() => setLoading(false));
  }, [url]);
}
```

```
    return { data, loading, error };  
  }
```

10:45 - 11:15 | EXERCISE: React with TypeScript

[SAY] "Write React components with TypeScript."

[PROJECT/WRITE]

```
// 1. Define Product interface  
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  description?: string;  
}  
  
// 2. ProductCard component  
const ProductCard: React.FC<{ product: Product }> = ({ product }) => {  
  // Your code:  
};  
  
// 3. ProductList component with state  
interface ProductListProps {  
  products: Product[];  
  onProductSelect?: (id: number) => void;  
}  
  
const ProductList: React.FC<ProductListProps> = ({ products, onProductSelect }) => {  
  // Your code:  
};
```

[GIVE 10 MINUTES]

[SOLUTION]

```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  description?: string;  
}  
  
const ProductCard: React.FC<{ product: Product }> = ({ product }) => {  
  return (  
    <div className="border p-4 rounded">  
      <h3 className="font-bold">{product.name}</h3>  
    </div>  
  );  
};
```

```

        <p className="text-gray-600">${product.price.toFixed(2)}</p>
        {product.description && <p className="text-sm">{product.description}</p>}
    </div>
  );
};

interface ProductListProps {
  products: Product[];
  onProductSelect?: (id: number) => void;
}

const ProductList: React.FC<ProductListProps> = ({ products, onProductSelect }) => {
  return (
    <div className="grid grid-cols-3 gap-4">
      {products.map(product => (
        <div
          key={product.id}
          onClick={() => onProductSelect?.(product.id)}
          className="cursor-pointer"
        >
          <ProductCard product={product} />
        </div>
      ))}
    </div>
  );
};

```

11:15 - 11:45 | TypeScript with Node.js

[SAY] "TypeScript works great with Node.js for backend development."

[WRITE ON WHITEBOARD]

Express with TypeScript

```

import express, { Request, Response, NextFunction } from 'express';

interface CreateUserRequest {
  name: string;
  email: string;
  age: number;
}

interface UserResponse {
  id: number;
  name: string;
  email: string;
  age: number;
  createdAt: Date;
}

```

```

    }

    const app = express();
    app.use(express.json());

    // GET endpoint
    app.get('/api/users', (req: Request, res: Response) => {
        const users: UserResponse[] = [
            { id: 1, name: 'Alice', email: 'alice@example.com', age: 25, createdAt: new Date() };
            res.json(users);
        });

    // POST endpoint
    app.post('/api/users', (req: Request<{}, {}, CreateUserRequest>, res: Response) => {
        const { name, email, age } = req.body;

        if (!name || !email || !age) {
            return res.status(400).json({ error: 'Missing required fields' });
        }

        const newUser: UserResponse = {
            id: Date.now(),
            name,
            email,
            age,
            createdAt: new Date()
        };

        res.status(201).json(newUser);
    });

    // Error handling
    app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
        console.error(err.stack);
        res.status(500).json({ error: 'Something went wrong!' });
    });

```

11:45 - 12:15 | EXERCISE: Node.js with TypeScript

[SAY] "Create a Node.js API with TypeScript."

[PROJECT/WRITE]

```

// 1. Setup project
// npm init -y
// npm install typescript express @types/node @types/express --save-dev

```

```
// 2. Create tsconfig.json
// 3. Create src/index.ts with:
// - Express server
// - GET /items endpoint
// - POST /items endpoint
// - DELETE /items/:id endpoint

// 4. Define Item interface
interface Item {
  // Your code:
}

// 5. Implement CRUD operations
// 6. Run the server
```

[GIVE 10 MINUTES]

[SOLUTION]

```
import express, { Request, Response } from 'express';

interface Item {
  id: number;
  name: string;
  description?: string;
  createdAt: Date;
}

let items: Item[] = [];
let nextId = 1;

const app = express();
app.use(express.json());

app.get('/api/items', (req: Request, res: Response) => {
  res.json(items);
});

app.post('/api/items', (req: Request, res: Response) => {
  const { name, description } = req.body;

  if (!name) {
    return res.status(400).json({ error: 'Name is required' });
  }

  const newItem: Item = {
    id: nextId++,
    name,
    description,
    createdAt: new Date()
  };
};
```

```
    items.push(newItem);
    res.status(201).json(newItem);
  });

app.delete('/api/items/:id', (req: Request, res: Response) => {
  const id = parseInt(req.params.id);
  const index = items.findIndex(item => item.id === id);

  if (index === -1) {
    return res.status(404).json({ error: 'Item not found' });
  }

  items.splice(index, 1);
  res.json({ success: true });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Afternoon Session Script

13:00 - 14:00 | EXERCISE: Complete TypeScript Project

[SAY] "Build a complete TypeScript application."

[PROJECT/WRITE]

Final Project: Task Management System

Requirements:

1. Task interface with id, title, completed, createdAt
2. TaskService class with CRUD operations
3. TaskController with CLI commands
4. Main program with user interface
5. Full TypeScript typing

Features:

- Add task
- List tasks (all or incomplete)
- Complete task
- Delete task
- Save/Load from file (bonus)

[GROUP EXERCISE - 60 minutes]

Teams of 3-4. Build complete system.

[FACILITATOR SOLUTION]

```
// types.ts
export interface Task {
  id: number;
  title: string;
  description?: string;
  completed: boolean;
  createdAt: Date;
  completedAt?: Date;
}

// service.ts
export class TaskService {
  private tasks: Task[] = [];
  private nextId = 1;

  addTask(title: string, description?: string): Task {
    const task: Task = {
      id: this.nextId++,
      title,
      description,
      completed: false,
      createdAt: new Date()
    };
    this.tasks.push(task);
    return task;
  }

  getAllTasks(): Task[] {
    return this.tasks;
  }

  getIncompleteTasks(): Task[] {
    return this.tasks.filter(task => !task.completed);
  }

  getTaskById(id: number): Task | undefined {
    return this.tasks.find(task => task.id === id);
  }

  completeTask(id: number): Task | undefined {
    const task = this.getTaskById(id);
    if (!task) return undefined;

    task.completed = true;
    task.completedAt = new Date();
    return task;
  }

  deleteTask(id: number): boolean {
    const index = this.tasks.findIndex(task => task.id === id);
    if (index === -1) return false;
  }
}
```

```

        this.tasks.splice(index, 1);
        return true;
    }

    getStats(): { total: number; completed: number; incomplete: number } {
        const total = this.tasks.length;
        const completed = this.tasks.filter(t => t.completed).length;
        return { total, completed, incomplete: total - completed };
    }
}

// controller.ts
import readline from 'readline';

export class TaskController {
    private service: TaskService;
    private rl: readline.Interface;

    constructor(service: TaskService) {
        this.service = service;
        this.rl = readline.createInterface({
            input: process.stdin,
            output: process.stdout
        });
    }

    start(): void {
        console.log('\n📁 Task Manager');
        console.log('=' .repeat(40));
        this.showHelp();
        this.prompt();
    }

    private prompt(): void {
        this.rl.question('\n> ', (input: string) => {
            const [command, ...args] = input.trim().split(' ');
            const arg = args.join(' ');
            this.handleCommand(command, arg);
            this.prompt();
        });
    }

    private handleCommand(command: string, arg: string): void {
        switch (command.toLowerCase()) {
            case 'add':
                this.addTask(arg);
                break;
            case 'list':
                this.listTasks();
                break;
            case 'complete':
                this.completeTask(parseInt(arg));
                break;
        }
    }
}

```

```

        case 'delete':
            this.deleteTask(parseInt(arg));
            break;
        case 'stats':
            this.showStats();
            break;
        case 'help':
            this.showHelp();
            break;
        case 'exit':
            console.log('👋 Goodbye!');
            this.rl.close();
            process.exit(0);
        default:
            console.log(`❌ Unknown command: ${command}`);
            this.showHelp();
    }
}

private addTask(title: string): void {
    if (!title) {
        console.log('❌ Please provide a title: add "My task"');
        return;
    }
    const task = this.service.addTask(title);
    console.log(`✅ Added task: ${task.id} - "${task.title}"`);
}

private listTasks(): void {
    const tasks = this.service.getAllTasks();
    if (tasks.length === 0) {
        console.log('📭 No tasks');
        return;
    }
    console.log(`\n📋 Tasks:`);
    tasks.forEach(task => {
        const status = task.completed ? '✅' : '❌';
        const date = task.createdAt.toLocaleDateString();
        console.log(`${status} [${task.id}] ${task.title} (${date})`);
        if (task.description) {
            console.log(`    ${task.description}`);
        }
    });
}

private completeTask(id: number): void {
    if (isNaN(id)) {
        console.log('❌ Please provide a valid ID: complete 1');
        return;
    }
    const task = this.service.completeTask(id);
    if (task) {
        console.log(`✅ Task ${id} completed`);
    }
}

```

```

    } else {
        console.log(`❌ Task ${id} not found`);
    }
}

private deleteTask(id: number): void {
    if (isNaN(id)) {
        console.log(`❌ Please provide a valid ID: delete 1`);
        return;
    }
    const success = this.service.deleteTask(id);
    if (success) {
        console.log(`🗑️ Task ${id} deleted`);
    } else {
        console.log(`❌ Task ${id} not found`);
    }
}

private showStats(): void {
    const stats = this.service.getStats();
    console.log(`\n📊 Statistics:`);
    console.log(`    Total: ${stats.total}`);
    console.log(`    Completed: ${stats.completed}`);
    console.log(`    Incomplete: ${stats.incomplete}`);
}

private showHelp(): void {
    console.log(`\n📖 Commands:`);
    console.log(`    add "title"      - Add a new task`);
    console.log(`    list             - List all tasks`);
    console.log(`    complete <id>    - Mark task as complete`);
    console.log(`    delete <id>     - Delete a task`);
    console.log(`    stats            - Show statistics`);
    console.log(`    help             - Show this help`);
    console.log(`    exit             - Exit the application`);
}
}

// index.ts - Main entry
import { TaskService } from './service';
import { TaskController } from './controller';

const service = new TaskService();
const controller = new TaskController(service);
controller.start();

```

14:00 - 15:00 | Project Presentations

[SAY] "Each team will present their project."

Presentation Structure (5-7 minutes each):

1. Overview (1 minute)

- What problem does your app solve?
- What features does it have?

2. Code Walkthrough (2-3 minutes)

- Show key TypeScript types/interfaces
- Show important functions or classes
- Explain how TypeScript helped

3. Demonstration (2 minutes)

- Show the app running
- Demonstrate key features

4. Q&A (1 minute)

- Answer questions from other teams

15:00 - 15:30 | FINAL WRAP-UP & CERTIFICATES

[SAY] "Congratulations on completing the TypeScript course!"

[WRITE ON WHITEBOARD - Full Summary]

TYPE SCRIPT FOUNDATIONS
DAY 1: BASIC TYPES - string, number, boolean, <code>null</code>, undefined - Type annotations, type inference - Arrays, tuples, functions
DAY 2: ADVANCED TYPES - Union <code>and</code> intersection types - Literal types, type aliases, interfaces - Type guards, type assertions
DAY 3: OOP WITH TYPESCRIPT - Classes, constructors, methods - Access modifiers: <code>public</code>, <code>private</code>, <code>protected</code> - Inheritance, abstract classes, polymorphism
DAY 4: GENERICS & UTILITY TYPES

- Generic functions, classes, interfaces
- Utility types: Partial, Pick, Omit, Record
- Mapped types, conditional types, infer

DAY 5: REAL-WORLD TYPESCRIPT

- TypeScript configuration
- React with TypeScript
- Node.js with TypeScript
- Complete project

KEY INSIGHT: TypeScript = JavaScript + Static Types
 Rule: Catch errors at compile time, **not** runtime.

[SAY] "Let's do a final Q&A."

Common Questions:

Q: "When should I use `any`?" A: "Never use `any` unless absolutely necessary. Use `unknown` if you must."

Q: "How do I choose between interface and type alias?" A: "Use interface for object shapes. Use type alias for unions and complex types."

Q: "What's the most important TypeScript feature?" A: "Type safety. Catching errors before runtime."

[DISTRIBUTE] Give each participant a completion certificate.

[FINAL CLOSING STATEMENT] "TypeScript is a powerful tool that makes JavaScript safer and more maintainable. The skills you've learned this week - types, generics, interfaces, classes - are applicable to any TypeScript project, whether frontend or backend."

"Remember: **TypeScript is JavaScript with types. It's your friend.**"

"Thank you for your attention. Any final questions?"

Day 5 Facilitator Cheat Sheet

Mistake	How to Fix
Forgetting to compile	"Run <code>tsc</code> before running code"
Wrong <code>outDir</code> path	"Check <code>tsconfig.json</code> path settings"
Not using <code>@types</code> packages	"Install type definitions for Node/Express"
Missing <code>--watch</code> flag	"Use <code>tsc --watch</code> for auto-compile"

Mistake	How to Fix
Runtime errors from type errors	"Typescript catches these at compile time!"

Appendix: Answer Keys

Day 1 Answers

Exercise 1.1: Type Annotations

```
let firstName: string = "John";
let age: number = 30;
let isActive: boolean = true;
let hobbies: string[] = ["reading", "gaming", "coding"];
let scores: number[] = [85, 90, 78];
let person: { name: string; age: number; city: string } = {
  name: "Alice",
  age: 25,
  city: "London"
};
```

Exercise 1.2: Functions

```
function double(x: number): number {
  return x * 2;
}

function greetUser(name: string, greeting?: string): string {
  return greeting ? `${greeting}, ${name}!` : `Hello, ${name}!`;
}

function sumArray(numbers: number[]): number {
  return numbers.reduce((total, num) => total + num, 0);
}

function createPerson(name: string, age: number): { name: string; age: number } {
  return { name, age };
}
```

Day 2 Answers

Exercise 2.1: Union Types

```
type ID = string | number;
```

```
function getMessage(id: ID): string {
  if (typeof id === "string") {
    return `User ID: ${id}`;
  } else {
    return `User ID: #${id}`;
  }
}

type Status = "loading" | "success" | "error" | "idle";

function getStatusMessage(status: Status): string {
  switch (status) {
    case "loading": return "Loading...";
    case "success": return "Operation successful!";
    case "error": return "An error occurred!";
    case "idle": return "Ready";
  }
}
```

Exercise 2.2: Type Aliases

```
type HttpMethod = "GET" | "POST" | "PUT" | "DELETE";

type User = {
  id: number;
  name: string;
  email: string;
  age?: number;
};

type DoubleFunction = (x: number) => number;

type Product = {
  readonly id: number;
  name: string;
  price: number;
  category: string;
  inStock: boolean;
};
```

Day 3 Answers

Exercise 3.1: Creating Classes

```
class Book {
  private title: string;
  private author: string;
  private pages: number;
  private isRead: boolean;
```

```

    constructor(title: string, author: string, pages: number) {
        this.title = title;
        this.author = author;
        this.pages = pages;
        this.isRead = false;
    }

    read(): string {
        this.isRead = true;
        return `You have read "${this.title}" by ${this.author}`;
    }

    getInfo(): string {
        const status = this.isRead ? "Read" : "Not read yet";
        return `"${this.title}" by ${this.author} - ${this.pages} pages - ${status}`;
    }

```

Exercise 3.2: Inheritance

```

class Vehicle {
    protected make: string;
    protected model: string;
    protected year: number;

    constructor(make: string, model: string, year: number) {
        this.make = make;
        this.model = model;
        this.year = year;
    }

    getInfo(): string {
        return `${this.year} ${this.make} ${this.model}`;
    }
}

class Car extends Vehicle {
    private doors: number;

    constructor(make: string, model: string, year: number, doors: number) {
        super(make, model, year);
        this.doors = doors;
    }

    getInfo(): string {
        return `${super.getInfo()} - ${this.doors} doors`;
    }
}

class Motorcycle extends Vehicle {

```

```
    private type: string;

    constructor(make: string, model: string, year: number, type: string) {
        super(make, model, year);
        this.type = type;
    }

    getInfo(): string {
        return `${super.getInfo()} - ${this.type}`;
    }
}
```

Day 4 Answers

Exercise 4.1: Generic Functions

```
function firstElement<T>(array: T[]): T | undefined {
    return array[0];
}

function reverseArray<T>(array: T[]): T[] {
    return array.slice().reverse();
}

function filterArray<T>(array: T[], predicate: (item: T) => boolean): T[] {
    return array.filter(predicate);
}

function findItem<T>(array: T[], predicate: (item: T) => boolean): T | undefined {
    return array.find(predicate);
}
```

Exercise 4.2: Generic Classes

```
class Queue<T> {
    private items: T[] = [];

    enqueue(item: T): void {
        this.items.push(item);
    }

    dequeue(): T | undefined {
        return this.items.shift();
    }

    peek(): T | undefined {
        return this.items[0];
    }
}
```

```

    size(): number {
        return this.items.length;
    }

    isEmpty(): boolean {
        return this.items.length === 0;
    }
}

class KeyValuePair<K, V> {
    constructor(private key: K, private value: V) {}

    getKey(): K { return this.key; }
    getValue(): V { return this.value; }
    setValue(value: V): void { this.value = value; }
}

```

Exercise 4.3: Utility Types

```

type OptionalUser = Partial<User>;
type UserContact = Pick<User, "id" | "name" | "email">;
type UserPublic = Omit<User, "password" | "address">;
type ReadonlyUser = Readonly<User>;
type UserPreferences = Record<string, boolean>;
type NonNullableString = NonNullable<string | null | undefined>;

```

Day 5 Answers

Exercise 5.1: Complete TypeScript Project

```

// Complete project solution provided in Day 5 afternoon session
// See "Final Project: Task Management System"

```

Appendix: TypeScript Setup Guide

Initial Setup Commands

```

# Install Node.js (if not installed)
# Download from https://nodejs.org/

# Install TypeScript globally
npm install -g typescript

# Check versions
node --version

```

```
tsc --version

# Create project
mkdir typescript-project
cd typescript-project
npm init -y

# Install TypeScript locally
npm install typescript --save-dev

# Create tsconfig.json
npx tsc --init

# Create source directory
mkdir src

# Create index.ts
echo "console.log('Hello TypeScript');" > src/index.ts

# Compile
npx tsc

# Run
node dist/index.js

# Watch mode
npx tsc --watch
```

VS Code Extensions

Recommended extensions:

- TypeScript and JavaScript Language Features (built-in)
- ESLint
- Prettier
- TypeScript Toolbox

Troubleshooting

Error: "tsc: command not found"

- Reinstall TypeScript: `npm install -g typescript`
- Or use `npx tsc`

Error: "Cannot find module"

- Install @types for the module: `npm install @types/module-name --save-dev`

Error: "File not found"

- Check path in tsconfig.json
- Verify file exists in source directory

END OF FACILITATOR EDITION

Course Completion Certificate

This certifies that

has successfully completed the

5-Day TypeScript Programming Foundations Course

covering all fundamental concepts including basic types, functions, interfaces, classes, generics, utility types, and real-world TypeScript applications.

Date: _____

Instructor: _____

Signature: _____

END OF COMPLETE COURSE BOOKLET